Applied Software Development Lab
Lab Handout 8: Things get vaporized

Learning objectives
- **Deepening object-oriented programming**
- **Refactoring existing programs**
- **Incremental software development: small changes, testing, and versioning**

Step 1: Cloning your own repository

Today, too, we are working "only" with Python. The goal is to add new features. For example, balls that you miss with the paddle should disappear at the bottom edge. And – as you would expect from Breakout – there should be stones (bricks) that disappear when the ball hits them. The ball is reflected in the process.

Our project currently consists of only a few modules (`ball.py`, `breakout.py`, `controller.py`, `game.py`, `paddle.py`, `rocket.py`, `serial-reader.py`). Still, you should not underestimate how hard it is to keep track of a program that consists of several parts – even if each file is only a few lines long.

You need
- on the one hand, an eye for the **big picture**: What does each module do?
- on the other hand, an eye for **detail**: How does the module do what it is supposed to do?

You have to switch flexibly back and forth between these two levels. Beyond a certain level of complexity, keeping both in mind at the same time is neither possible nor necessary. This is the skill we want to train in the first step.

We proceed as follows: start the game, observe what it can do, then skim the code to recapitulate its structure.

Preparation: Opening two terminals
As always: left terminal for editing, right terminal for running commands.

Right terminal (cloning the repository and starting the game)
First, clone your repository with
`git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm`
Use `cd asp-ulm/python` to change into the Python directory, and then start the game with `python breakout.py`.

You should observe:
- The balls are cleanly reflected at all edges (top, bottom, left, right). More precisely: the **center** of the balls always stays within the drawing surface – this was taken into account in the reflection.

- You can make the ball bounce off the **paddle**. But "angle of incidence = angle of reflection" does **not** apply here:
  - If you hit the ball in the middle, it flies straight up.
  - Far left → 30° to the left.
  - Far right → 30° to the right.
- With the **up arrow key**, the paddle fires rockets (small discs) that fly straight up and disappear at the top edge.

Left terminal (examining the code)
Now the point is to understand **how** these features were implemented. There is no “right” or “wrong” here – that depends on style, taste, and also on the state of the project.

One possible way is the top-down approach:
1. Start with the main loop in `breakout.py`.
2. Among other things, you will see: there are classes such as `Ball` and `Rocket`, of which several instances are created, each with a `draw` and a `move` method.
3. There is also the class `Paddle`, of which there is only one instance – with (static) methods such as `left`, `right`, and `draw`.
4. Then you can go through the files `ball.py`, `rocket.py`, and `paddle.py` one by one.
5. While reading, keep asking yourself questions such as:
   - “Why did I do it this way back then?”
   - “What exactly does this or that mean in Python again?”
   - “How are the modules connected to each other?”

You may notice: some modules such as `controller.py` or `serial-reader.py` are **not used at all** when the game starts. This is not a mistake – these files concern the **serial connection**, which will later allow you to control the game with a **game controller** instead of the keyboard.

Today, however, it is **only** about the game features. It is perfectly sufficient if you roughly recall *what* these modules are meant for. You do not need to look at them in detail today.
One thing at a time.

Task
Go through this process on your own. Start the game, observe its behavior, and examine the code. If anything is unclear – ask!

Remark
While looking through the code, we came across a variant like this:

```
for r in rockets:
    r.draw()
    r.move()
    rockets = [item for item in rockets if item.alive]
```

In Python, this **seemingly works without problems**, even though the list `rockets` is **modified during the iteration**. The reason: in each pass of the loop, a **new list** is created and the name `rockets` is rebound – but the loop **keeps running over the original iterator**, which still refers to the old list. So it only works because Python allows **inefficient (and potentially confusing)** code.

```
for r in rockets:
    r.draw()
    r.move()
rockets = [item for item in rockets if item.alive]
```

Here, **all rockets are first moved and drawn**, and **then** all rockets that are no longer visible (`alive == False`) are removed.

This is **more efficient** and **easier to understand** – also for your future self in a few weeks 😉

© Michael Lehn | CC BY-SA 4.0

Step 2: Balls you miss …

Now we test whether recapitulating in the first step has paid off.

Goal: If you miss a ball with the paddle, it should disappear as soon as it reaches the bottom edge of the drawing surface. Among other things, this means: the corresponding ball object should be removed from the list `balls`.

Task
Implement this feature:
- Detect when a ball is “lost” (e.g., based on its y-position).
- Remove the corresponding object from the list `balls`.

Tip: Think about **how this was done for the rockets**. There, too, objects disappear when they reach the edge. The goal is to understand this mechanism and **transfer it to the balls in the same way**.

If you have **questions**, let us know. If you think everything works, **show us** what you have changed. **Once we give you the OK**, off to the repository:

```
git commit -m "ball disappears"
git push
```

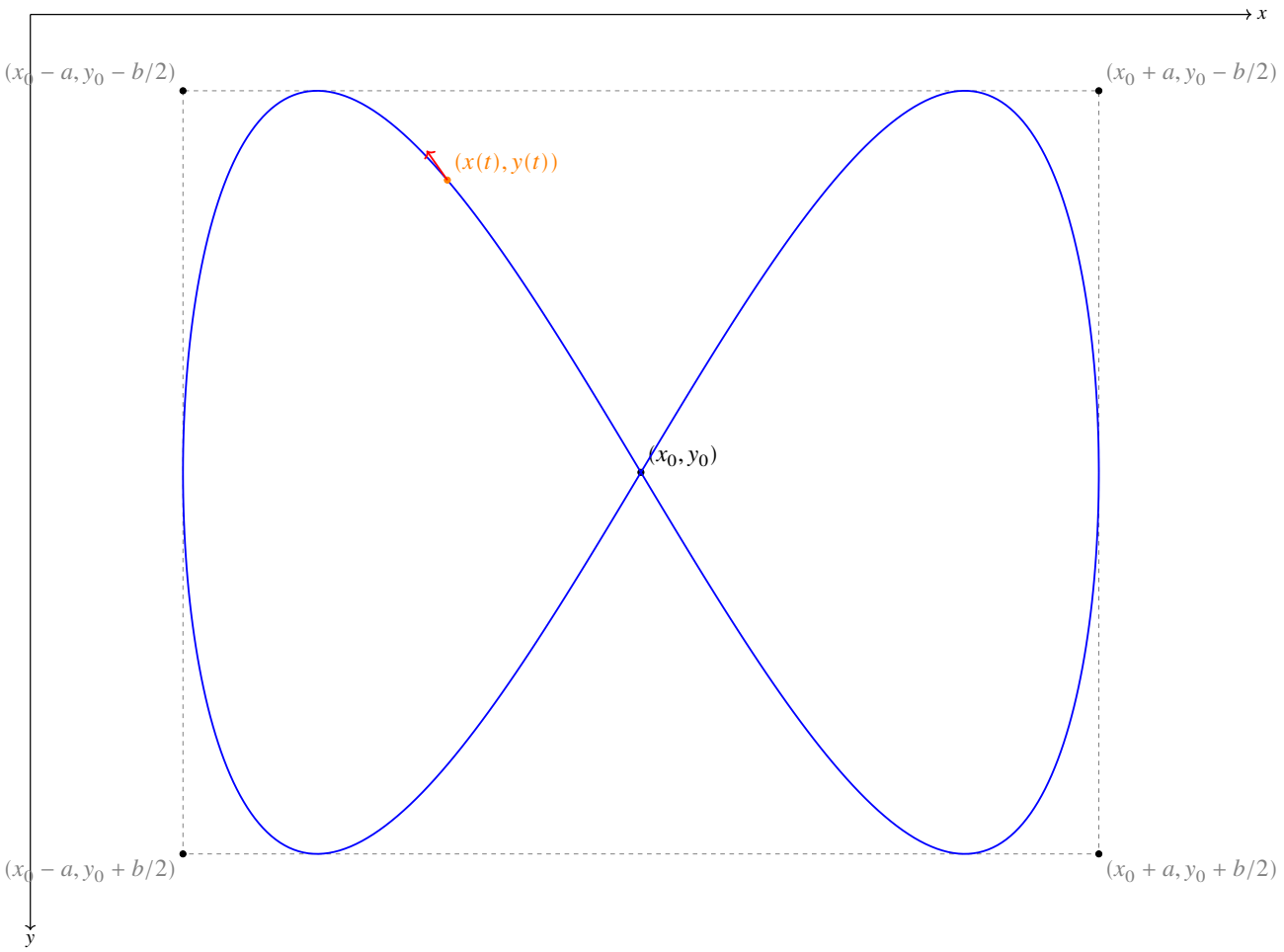
© Michael Lehn | CC BY-SA 4.0

Step 3: Space Invaders in Breakout?

Why not? This is an invasion in the truest sense of the word – and at the same time a good moment to notice how much it pays off to have looked at the existing code beforehand.

Idea: Analogous to the class `Ball`, you can now program a class `Ship`.
Each object of this class represents a spaceship that moves across the drawing surface – just like the “invaders” in the classic game.

Movement along a curve
Of course, the spaceships should **not simply move in a straight line** – that would make them too easy to hit. Instead, you can let them fly along an interesting curve. A nice option is the so-called **lemniscate of Gerono**. The trace of this curve looks like a **figure eight lying on its side**:



The curve is parametrized by time (with t as the time) and reads:

$$x(t) = x_0 + a \cdot \cos t, \quad y(t) = y_0 + \frac{b}{2} \cdot \sin 2t$$

This means: the spaceship moves within a rectangle with the corners:

- **Top left:** $(x_0 - a, y_0 - b/2)$
- **Bottom left:** $(x_0 - a, y_0 + b/2)$
- **Bottom right:** $(x_0 + a, y_0 + b/2)$
- **Top right:** $(x_0 + a, y_0 - b/2)$

© Michael Lehn | CC BY-SA 4.0

Task: Bringing spaceships into the game

Write a new class `Ship` – analogous to the class `Ball`. A spaceship moves along a lemniscate and can (later) be shot down.

Part 1: Creating the class Ship
Create a new file `ship.py` and define a class `Ship` in it.
Since this class is very similar to the class `Ball`, you can simply start with a copy:

```
cp ball.py ship.py
```

Then modify the file step by step:

(A) Constructor

A spaceship should be created like this:

```
ship = Ship(0)
```

The parameter `0` stands for the initial value of the time $t = 0$.

The class should have the following attributes:

```
self.x0 = Game.width / 2
self.y0 = Game.height / 3
self.a = 0.9 * Game.width / 2
self.b = 0.9 * Game.height / 3
```

In addition, you need:

```
self.t = t                    # current value of the time
self.alive = True             # False means shot down
self.x, self.y = None, None # current position (set in move)
self.move()                   # call move to set the position
```

(B) Method `move(self)`

Here you compute the new position along the lemniscate:

```
def move(self):
    self.x = self.x0 + self.a * math.cos(self.t)
    self.y = self.y0 + self.b * math.sin(2 * self.t) / 2
    self.t += 0.01
    while self.t > 2 * math.pi:
        self.t -= 2 * math.pi
```

Resetting t to the interval $[0, 2\pi)$ is **not strictly necessary** – Python has no problem with numbers growing arbitrarily large (and don't worry: in this case, the memory will not fill up in your lifetime 😄). But: it is more elegant and saves memory – and it makes a true engineer's heart beat faster ✅

(C) Method `draw(self)`

Simply draw the spaceship as a disc with a larger radius than the ball – and preferably in a different color.

Part 2: Building the spaceships into the game
Now the spaceships have to be integrated into `breakout.py` – analogous to the balls:

1. Importing the class: `from ship import Ship`
2. Creating a list of ships, e.g., `ships = [Ship(0), Ship(0.5), Ship(1)]`
3. Moving, drawing, and removing ships that have been shot down from the list

Once your code runs:

```
git add ship.py
git commit -a -m "spaceships!"
git push
```

© Michael Lehn | CC BY-SA 4.0

Step 4: Now it's getting serious!

Of course, you should be able to **shoot down a spaceship** with a **ball** or a **rocket**. This works by checking whether the distance between a spaceship and a ball or a rocket is **small enough**.

Here is an example of how to check whether a **ball** has come too close to a **spaceship**:

```
for s in ships:
    for b in balls:
        dist = math.hypot(b.x - s.x, b.y - s.y)
        if dist <= 20:
            s.alive = False
```

What exactly happens here?
We want to find out whether a **ball has hit a spaceship**. To do so, we have to calculate, for **every spaceship**, the distance to **every ball**. For this purpose, two loops are **nested inside each other**:

- The **outer loop** runs with the variable `s` over all spaceships in the list `ships`.
- For **each individual value of** `s`, the **inner loop** runs with the variable `b` over all balls in the list `balls`.

So in the **innermost loop**, there is always exactly **one spaceship** `s` and one ball `b` – for every possible pair of spaceship and ball.
Then the distance between these two points is calculated with good old Pythagoras:

$$\text{dist} = \sqrt{(\text{b.x} - \text{s.x})^2 + (\text{b.y} - \text{s.y})^2}$$

If the distance is less than or equal to 20, the spaceship has been hit. Why 20? In my version, a ball has radius 5 and a spaceship has radius 15.

Task:

1. **Import** the module `math` in `breakout.py`, if you have not done so yet:

   ```
   import math
   ```

   Insert the above code at a suitable place in the main loop:
   - **after** all objects have been moved and drawn
   - **before** the lists `balls` and `ships` are cleaned up (i.e., the "corpses" are removed)

   Test whether you can now shoot down a spaceship with a ball.
2. Actually, the point of the rockets was to be able to shoot something down with them.
   Change the code so that the inner loop iterates over the list `rockets` instead of `balls`. **Test** whether you can now shoot down a spaceship with a rocket.
3. It would of course be even better if you could shoot down a spaceship **with both a ball and a rocket** – without duplicate code.
   An obvious approach would be, for example, the following:

   ```
   for b in [balls, rockets]:
       …
   ```

   You might think that this iterates over **all balls and all rockets**. **But convince yourselves that this does not work!**

   So what really happens here?
   `[balls, rockets]` is a list that contains exactly two elements: namely the list `balls` and the list `rockets`. This means: the loop runs twice – once with `b = balls`, once with `b = rockets` – and `b` is then a whole list each time, not a single object.

© Michael Lehn | CC BY-SA 4.0

4. With `balls + rockets`, you create a new list that contains all elements of `balls` and of `rockets`.

   So if you write:

   ```
   for s in ships:
       for b in balls + rockets:
           dist = math.hypot(b.x - s.x, b.y - s.y)
           if dist <= 20:
               s.alive = False
   ```

   then everything should work as intended: you can vaporize a spaceship with a ball as well as with a rocket 💥

   Test whether this works. If so – then you know that the effort has paid off. And then, as always, into the repository with it:

   ```
   git commit -a -m "shoot to kill"
   git push
   ```

Step 5: Bricks!

Now a **wall of stones (bricks)** enters the game – which of course makes sense if the game is called **Breakout** 😉

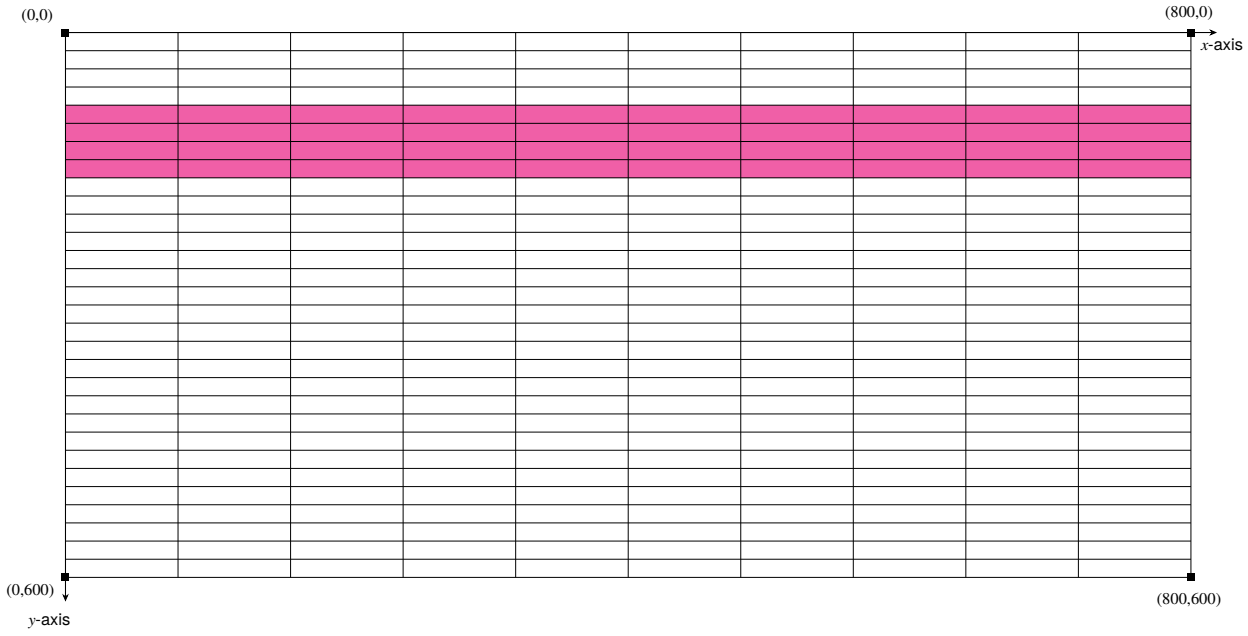
In a **first version**, the balls should simply still **fly through the bricks**. At first, the point is **not** the collision, but rather:

🧩 **Where on the drawing surface is there a brick – and where not?**

The grid

For simplicity, a **brick** in our case is **80 pixels wide** and **20 pixels high**. Since the drawing surface is **800 pixels wide** and **600 pixels high**, this results in a **grid with 10 columns and 30 rows**. Of course, not every cell in this grid is occupied by a brick.

For example, only a few of the rows could contain bricks at first – as in the following sketch:



A bit of mathematics

A **grid** with **10 columns** and **30 rows**, in which each cell either contains a brick or not, can be thought of as a **matrix**.
Each entry is then either `1` → brick present, or `0` → no brick present. And because we, as mathematicians, value precision and conventions: we speak of a **30 × 10 matrix** (rows × columns).

Implementation in Python: `numpy`

In Python, the module `numpy` lets you work with matrices very efficiently. For our purposes, we only need:
- a **zero-initialized 30 × 10 matrix**.
- **Indexing** starts **at 0**: the row index runs from 0 to 29, the column index from 0 to 9.
- access to individual entries for **reading and writing**.

The class `Wall`

Create a new file `wall.py` and write the following code in it:

```
 1 import pygame
 2 import numpy
 3 from game import Game
 4
 5 class Wall:
 6     brick = numpy.zeros((30, 10))
 7
 8     def init():
 9         for i in range(4, 8):
10             for j in range (0, 10):
11                 Wall.brick[i, j] = 1
12
13     def draw():
14         for i in range(0, Wall.brick.shape[0]):
15             for j in range(0, Wall.brick.shape[1]):
16                 if Wall.brick[i, j] == 0:
17                     continue
18                 w = 80
19                 h = 20
20                 x = j * w
21                 y = i * h
22                 r = (x + 1, y + 1, w - 1, h - 1)
23                 pygame.draw.rect(Game.screen, (223, 105, 165), r)
```

Explanation of `wall.py`

- In **line 2**, the module `numpy` is imported.
- The class has a **class attribute** `brick`, which is a **30 × 10 matrix full of zeros**.
- The method `init` sets all matrix entries in **rows 4 to 7** to `1` – that is where the bricks are!
- The method `draw` draws the wall:
  - With `Wall.brick.shape[0]` and `Wall.brick.shape[1]`, you query the number of rows and the number of columns, respectively.
  - Two nested loops run through the matrix.
  - `i` stands for the **row index**, `j` for the **column index**.
  - If the entry is `0`, `continue` jumps to the next loop iteration.
  - Otherwise, a rectangle is drawn:
    - width `w = 80`, height `h = 20`
    - top left corner at `x = j * w`, `y = i * h`
    - slightly shifted (`+1`) and reduced in size (`-1`) for visible gaps

© Michael Lehn | CC BY-SA 4.0

Tasks:

1. **Build the brick wall into the game** by integrating the class Wall into `breakout.py`:
   (A) Import the class at the beginning of the file: `from wall import Wall`
   (B) After `Game.init()`, also call the initialization of the wall: `Wall.init()`
   (C) Within the main loop, also call `Wall.draw()` – preferably **after the balls have been drawn**: Wall.draw()
2. **Test** whether the brick wall is displayed as expected.
   If everything works – into the repository:
   ```
   git add wall.py
   git commit -a -m "the wall is up"
   git push
   ```

Step 6: Now we clear the bricks out of the way!

The bricks are currently pure decoration. In this step, they are to fulfill their purpose: **they are to be smashed by the balls.**

Of course, one could calculate exactly **whether and where a ball hits a brick**: after all, for each frame we know the old and the new position of a ball. From this, one could calculate which edge of a brick is crossed.

💡 Anyone who feels like a little math puzzle is welcome to “solve for x” on their own.

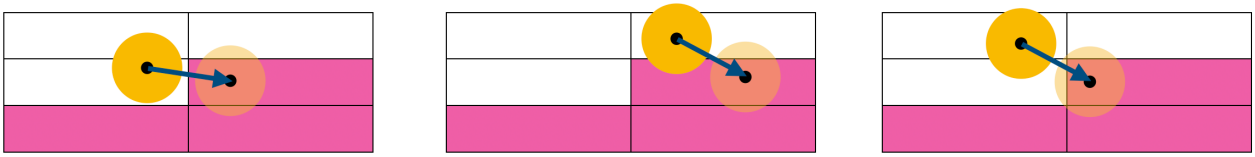
We make it easier for ourselves at this point – with an **approximation**:

- From the **old position** `x1`, `y1` of the ball, we calculate **in which grid cell** the ball was before: this gives a pair `i1`, `j1` (row, column).
- From the **new position** `x2`, `y2`, we calculate accordingly which **grid cell** the ball has now entered: this gives a pair `i2`, `j2` (row, column).

If the ball moves **from one cell into another cell with a brick**, we assume that a brick was smashed in the process – and **set the entry in the matrix** `Wall.brick[i2, j2]` to 0.

And there is more: We can even estimate approximately **how** the brick was hit:
- If `i1 ≠ i2`, the **top or bottom edge** was probably smashed through.
- If `j1 ≠ j2`, the **left or right edge** was probably smashed through.
- If both `i1 ≠ i2` and `j1 ≠ j2` hold, a corner would even have to have been hit, which, however, is rather rare in practice. Such cases also clearly show the limits of this approximation.

The following sketch shows three typical cases – and helps you get a feeling for how well (or how poorly) this method works in practice:



© Michael Lehn | CC BY-SA 4.0

How do you calculate the column and row in the grid from the (x, y) coordinates?

- If you divide the **x-coordinate** by the **width** of a cell and round the result down to an integer, you get the **column**. In Python, this is easily done with integer division (`//`).
- The same applies to the **y-coordinate**: if you divide it by the **height** of a cell and round down, you get the **row**.

Add the following class method to the class Wall:

```
13      def collision(x2, y2):
14          j2 = int(x2) // 80
15          i2 = int(y2) // 20
16          if i2 >= 0 and i2 < 30 and j2 >= 0 and j2 < 10:
17              if Wall.brick[i2, j2]:
18                  Wall.brick[i2, j2] = 0
```

What does this method do?

- In this version, `collision` is initially passed only the **new position** `(x2, y2)` of the ball.
- In **lines 14 and 15**, the column `j2` and the row `i2` are calculated from `x2` and `y2` – by integer division by the cell width and cell height, respectively.
- In **line 16**, we check whether `i2` and `j2` are valid indices in the grid at all – because the ball may already be **outside** the drawing surface (e.g., before the correction at the walls).
- In **line 17**, we check whether **there is a brick in the cell that was hit**.
- In **line 18**, this brick is then removed – so we "vaporize" the brick by setting the matrix entry to `0`.

Tasks:

1. In the `move` method of the class `Ball`, we now want to check whether the ball hits a brick. Only a few changes in `ball.py` are needed for this:
   (A) The class `Wall` has to be imported: `from wall import Wall`
   (B) In the `move` method (after `new_x` and `new_y` have been calculated and corrected), call the collision detection: `Wall.collision(new_x, new_y)`
   Test whether bricks now disappear on contact with a ball. The ball should fly through the bricks like a knife through butter – because its course currently remains completely unaffected. We only check whether it hits a brick – and then remove that brick.
2. **Now comes today's grand finale.**
   On contact with a brick, the ball should not only make the brick disappear but also bounce off. To do this, change the call in the `move` method of the class `Ball` to:
   ```
   self.vx, self.vy = Wall.collision(self.x, self.y, new_x, new_y,
                                     self.vx, self.vy)
   ```
   Now the old position (`self.x`, `self.y`), the new position (`new_x`, `new_y`), and the current velocity vector (`self.vx`, `self.vy`) are passed.
   The method should now return a (possibly adjusted) velocity vector. Now it is your turn to extend `Wall.collision(...)` accordingly:
   1. If the **top or bottom edge** of a brick is smashed through, the **y-component** of the velocity vector should be "reversed".
   2. If the **left or right edge** of a brick is smashed through, the **x-component** of the velocity vector should be "reversed".
   3. If **both indices** change, the ball bounces off a corner – and **both components** should be negated.

   Test whether the ball now bounces off correctly and removes the brick. If it all works:
   ```
   git commit -a -m "bricks can take a hit now"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0